CS101: Week 5 - From Ideas to Instructions

Lesson 1: Balancing Planning and Coding

Software development requires balancing upfront design with active programming. While rushing into code without preparation triggers logical bugs and messy architectures, over-planning risks stalling momentum. Creating a brief, high-level structural outline before typing keeps your program's flow transparent and significantly reduces execution rewrites.

```
# Plan: For each number in list, double value and append
'F' string modifier
nums = [2, 4, 6]
result = []
for n in nums:
    result.append(str(n * 2) + "F")
assert len(result) == 3 # Lightweight postcondition
validation
```

The Cost-Benefit Matrix of Pre-Planning

Pre-planning provides major structural benefits: it makes your program's execution flow clear, reduces code rewrites, helps team collaboration, and highlights edge cases early. However, excessive planning introduces operational costs, including upfront analysis delays and over-design risks. Assess the code's size, logic complexity, and team size to determine if a full plan is required before starting.

Lesson 2: Breaking the Problem

Complex programming requirements become easy to manage when broken down into ordered, plain-English pseudocode steps. This process extracts structural milestones into a clear top-to-bottom checklist, ensuring that every logical dependency, system variable initialization, and baseline input validation check is mapped out sequentially before any code is written.

```
# Ordered Plan Checklist:
# 1. Initialize data list and target tracking variable
# 2. Iterate through each list element
# 3. If value matches the condition, set target variable
and stop the loop
# 4. If search parameters end without a match, resolve
fallback state
# 5. Output the final result
```

The Rule of Atomic Pseudocode Actions

When drafting your planning checklists, each step must represent a single, atomic action. Avoid combining multiple logical decisions (such as "and" or "or" criteria) into a single line. Splitting operations—such as isolating input integer checks from numerical value limits—ensures that every verification gate remains distinct and easy to convert into standard code.

Lesson 3: Hierarchy in Pseudocode

Complex algorithms often involve nested operations, where certain sub-steps only occur under specific parent criteria. Representing these dependency paths in planning requires establishing a clear visual hierarchy. By utilizing structural decimal notation (such as step 1.1, 1.1.1) and applying 2-space indentation levels, nested execution layers become instantly scannable.

```
# 1. Start loop through dataset collections
#   1.1 If active element matches target condition
#       1.1.1 Execute target processing calculations
#       1.1.2 Terminate loop cycle immediately to
preserve tracking state
# 2. Output final calculation matrix metrics
```

Positioning Sentinel State Variables

Maintaining architectural hierarchy is essential for positioning your tracking "sentinel" variables (such as setting a search variable to `None` before initiating a lookup loop). Placing initialization variables outside the nested loops prevents them from being wiped out or reset during subsequent iteration loops.

Lesson 4: Turning Rules into Code

Converting business requirements into code requires mapping conditions to structured conditional blocks (`if/elif/else`). Maintaining rule consistency relies on writing precise boundary filters that match both upper and lower limits, ensuring that every input lands into exactly one mutually exclusive category.

```
# Sequential age boundary processing
if age >= 0 and age <= 12:
    categories.append("child")
elif age >= 13 and age <= 17:
    categories.append("teen")
elif age >= 18 and age <= 64:
    categories.append("adult")
else:
    categories.append("senior")
```

The Risk of Incomplete Condition Bounds

Omitting explicit upper or lower parameter bounds in conditional chains creates overlapping logic. For example, using a loose filter like `if age >= 18` inside a multi-tier pricing script can inadvertently catch senior citizen rows, short-circuiting downstream branches and corrupting your outputs.

Lesson 5: Sketching Flow

Before implementing a complex rules engine, mapping conditions to explicit outcomes using clean, non-coded flow strings keeps your logic stable. A high-quality logic system must satisfy two core structural traits: it must be **complete** (all potential inputs are accounted for) and **exclusive** (no single input matches more than one rule row).

```
# Condition Matrix Flow Mapping:
# Score range >= 0  and <= 59  --> Map to "fail" outcome
# Score range >= 60 and <= 89  --> Map to "pass" outcome
# Score range >= 90 and <= 100 --> Map to "honors"
outcome
```

Stress Testing Logic Flow Borders

Verifying a flow map requires dry-running test values right at the borders of each condition row (such as evaluating scores 59, 60, 89, and 90). If a boundary input triggers multiple rules or fails to find a valid branch, the flow contains logic errors that must be fixed before coding.

Lesson 6: Understanding Common Solution Patterns

As software systems expand, long, monolithic `if/elif/else` chains become difficult to read and maintain. The **lookup table pattern** resolves this by mapping fixed conditional keys directly to target values using a dictionary data structure, allowing code routines to execute values instantly in a single line.

```
# Lookup table dictionary pattern with safe fallback
handling
fee_map = {"MON": 10, "TUE": 12, "FRI": 8}
active_fee = fee_map.get("SAT", 0) # Safely defaults to
0; avoids KeyError
total_bill = active_fee * 1.10     # Applies 10%
calculated system tax
```

Defensive Coding with Dictionary .get() Lookups

Accessing a dictionary directly using square brackets (such as `fee_map["SAT"]`) will cause an immediate `KeyError` crash if the key is missing. Using the `.get(key, default)` pattern provides defensive validation by smoothly returning a fallback value when checking edge inputs, empty strings, or `None` types.

Lesson 7: Reviewing the Plan

Reviewing software plans requires evaluating logic through a strict structural lens to discover hidden bugs early. Applying formal logic reviews uncovers common programmatic gaps, including missing system initializations, unhandled edge boundaries, unreachable code rows, and overlapping condition branches.

```
# Corrected, mutually exclusive workflow paths
if number < 10:
    print("Under threshold range parameters")
if number >= 10:
    print("Meets or exceeds threshold range parameters")
```

Using the GAP-O Review Methodology

The GAP-O methodology is a framework for identifying structural bugs: **G**uard criteria present? **A**ll system branches reachable? **P**ostconditions covered completely? **O**verlaps or duplicate actions isolated? Running this check before writing code prevents unexpected bugs from slipping through.

Lesson 8: From Plan to Working Machine

Translating an abstract plan into code is most successful when done incrementally, row by row. Start by converting your pseudocode lines directly into numbered comment strings to create a script skeleton. From there, implement the actual code right underneath each comment, running lightweight verification tests at every step.

```
nums = [2, 4, 7]
# 1. Initialize tracking variable container to zero
total = 0
# 2. Establish loop cycle to process data array elements
for val in nums:
    # 3. Apply conditional filter rule to check for even
numbers
    if val % 2 == 0:
        total += val # Safe incremental state change
```

The Practice of Micro-Step Compilation Checks

Writing an entire program before running it makes bugs difficult to isolate. By utilizing Python's temporary `pass` statement inside newly created loop and conditional blocks, you can run and compile your script skeleton step-by-step to confirm your structure remains valid before writing the inner math logic.

Lesson 9: Keeping Comments That Connect to Pseudocode

Maintaining structural alignment between your initial design and your final source file requires keeping your pseudocode steps as active code documentation. This provides a clear structural trace, making it easy to see where each design rule is handled and ensuring that comments don't become outdated during subsequent updates.

```
# 1. Get favorite number input from user channel
user_entry = input("Enter your favorite number: ")
# 2. Apply a triple scaling factor calculation routine
scaled_total = int(user_entry) * 3
# 3. Output the scaled total to the user screen
print(scaled_total)
```

Maintaining Comment-Code Synchronization

When changes are made to your code logic (such as switching an equation from doubling to tripling a value), you must update its corresponding comment line immediately. Keeping comments and code synchronized prevents documentation drift, which can mislead future maintainers during software audits.

Lesson 10: Creating Function Stubs for Structure

Large, multi-tiered software applications should be mapped out using clear architectural layouts before filling in the core calculations. Creating **function stubs** allows you to outline your script's structural components, parameter expectations, and execution sequence while keeping the script fully runnable.

```
def load_data():
    pass # Structural data collection placeholder

def clean_data(some_list):
    pass # Processing parameters placeholder

def show_stats(total, count):
    pass # Output calculation visualization placeholder
```

Documenting with Purpose-Driven Explanations

Add a clear, intent-revealing comment starting with the phrase "# because..." directly above your function stubs. This documents the underlying architectural reason for the block's existence, explaining why it is required for the broader system pipeline rather than just stating what the code does.

Lesson 11: Building Code Incrementally

Once your function stubs are laid out, upgrade them from basic `pass` placeholders to use **safe dummy return values** (such as `return []` or `return 0`). This approach allows you to connect separate processing steps into a continuous pipeline and verify the data flow without risking system crashes.

```
def load_data():
    return [10, 20, 30] # Safe list dummy payload

def show_stats(total, count):
    if count == 0:
        return 0 # Defensive divide-by-zero validation
guard
    return total / count
```

The Mechanics of the Safe Stub Upgrade

Using consistent, primitive data placeholders (empty strings, lists, or integers) ensures that data chains remain stable during development. This method allows you to safely swap out individual placeholders for real calculations one-by-one, knowing that the rest of your pipeline remains fully testable.

Lesson 12: Creating Test Tables for Pseudocode Plans

A test table is a structured, comment-based layout that maps sample inputs directly to their expected outputs. Designing these verification matrices before writing core algorithms provides a clear blueprint for confirming that your final code works exactly as intended under normal and extreme conditions.

```
# TEST DRIVEN VALIDATION TABLE:
# input_val  | expected_output  | classification
# 5          | False            | Odd Entry Normal Test
# 0          | True             | Zero Entry Lower Edge
Test
# -3         | False            | Negative Entry Bound
Test
assert is_even(0) == True
assert is_even(5) == False
```

Reading the Meaning of Test Silence

When running a test suite built with lightweight `assert` statements, a successful pass produces no console output. This "silent pass" confirms that all execution behaviors match your expected test targets, while any deviation instantly triggers an `AssertionError` to reveal the bug location.

Lesson 13: Dry Running Plans with Test Values

Dry running is the practice of manually walking through your pseudocode steps using specific test inputs before running the code on a machine. This manual trace checks your conditional paths, tracks variable mutations, and catches logic issues or off-by-one errors before they reach production code.

```
def abs_value(n):
    if n < 0:
        n = -n # Flips negative values to positive
    print("trace after flip:", n) # Diagnostic tracing
block
    return n
assert abs_value(-5) == 5
```

Isolating Implementation Logic from Initialized Parameters

A common mistake when translating a dry run into real code is accidentally hardcoding your sample test values inside the body of the function (such as writing `n = 0` at the top of a calculation block). Ensure your routines rely purely on incoming input parameters to maintain flexibility.

Lesson 14: Integrating Logic into a Full App

Building a complete, production-grade application means gluing three core components together seamlessly: defensive input validation blocks, core business calculations, and clean output formatting. Isolating these routines inside distinct functions keeps your software flexible and easy to maintain.

```
def add_tip(total, tip_rate):
    # Business logic layer: converts percent entries to
decimal scales
    return total + (total * (tip_rate / 100)), tip_rate /
100

def main():
    bill_amt = get_amount() # Validation input layer
    final, rate = add_tip(bill_amt, 20)
    show_bill(bill_amt, rate, final) # Clean formatting
output layer
```

Reviewing Variable Scope Enclosures

Variables declared inside a function's local scope (such as `grand_total` inside a `main()` block) are fully enclosed and private to that runtime block. Attempting to read or modify these fields globally outside their home function triggers a `NameError`, preventing variable corruption bugs.

Lesson 15: Demonstrating the Working App

Validating an integrated application at scale requires using automated, table-driven testing loops instead of manual checks. Storing your verification scenarios inside a list of structured tuples allows you to test multiple data combinations smoothly while keeping your testing framework clean and efficient.

```
# Table-driven automated test suite cases: (total,
tip_pct, expected_grand)
test_cases = [(40, 20, 48), (55, 35, 74.25), (100, 0,
100)]

for total, pct, expected in test_cases:
    grand_total, rate = add_tip(total, pct)
    assert grand_total == expected # Automated pipeline
check
    print("Test case verified: Okay")
```

The Architectural Value of DRY Test Frameworks

Adhering to the **DRY (Don't Repeat Yourself)** pattern by running test cases through structured loops eliminates code duplication. If your function signatures or calculation rules change later, you only need to update your data matrix in one single location, keeping your test suite stable and easy to maintain.